AI MASTERCLASS • SESSION 02

DATA, EDA & DATA CLEANING

Understanding your data before building AI models

DATA | EDA | CLEANING

Mansouri Ahmed | BrainerX School, Constantine

WHAT WE COVER TODAY

01

WHAT IS DATA?

WWWWW-H Framework — Who, What, When, Where, Why, How

02

EDA — EXPLORATORY ANALYSIS

Visualise before modelling. Univariate & Multivariate

03

VISUALIZATION TOOLS

Matplotlib & Seaborn — when to use each

04

DATA CLEANING

Missing values, wrong types, duplicates, outliers

05

HANDS-ON EXERCISES

Real datasets: Online Retail, Titanic, medical data

01

DATA

WHAT IS DATA?

The raw material of all AI systems.

Before writing a single line of Python,
you must understand your data deeply.

WHAT IS DATA?

Data is any recorded information that can be processed, analysed, and used to make decisions or train AI models.

STRUCTURED

Tables, spreadsheets, databases. Rows & columns. Easy to process.

UNSTRUCTURED

Text, images, audio, video. No fixed format. Harder to process.

TIME SERIES

Data ordered by time. Stock prices, weather, sensors.

SPATIAL

Location-based data. GPS, maps, geographic info.

Every app you use generates data — your clicks, your location, your voice, your face.

THE WWWWW-H FRAMEWORK

Before analysing any dataset, always ask these 6 questions

WHO

Who collected this data?
Who are the subjects?

e.g. A hospital, a company, a government

WHAT

What information is stored?
What do the columns mean?

e.g. Age, price, GPS coordinates

WHEN

What time period does it cover?
Is it still relevant?

e.g. 2009–2011, last 30 days

WHERE

Where was it collected?
Geographic or system origin?

e.g. UK-based company, global customers

WHY

What was the purpose?
Why does this data exist?

e.g. Inventory management, fraud detection

HOW

How was it collected?
Manual entry, sensors, scraping?

e.g. Sales system logs, IoT sensors

EXERCISE — ONLINE RETAIL 2 DATASET

🎁 Online Retail 2

Kaggle / UCI Repository

Real UK e-commerce data
Dec 2009 – Dec 2011
541,909 rows × 8 columns

InvoiceNo	Invoice number
StockCode	Product code
Description	Product name
Quantity	Units ordered
InvoiceDate	Date & time
UnitPrice	Price per unit
CustomerID	Anonymized ID
Country	Customer country

WHO
An online retail company (UK-based)

WHAT
Each row = line-item in an invoice. InvoiceNo, Quantity, UnitPrice, CustomerID…

WHEN
December 2009 to December 2011

WHERE
UK business — but customers are global worldwide

WHY
Inventory management, accounting, and order tracking

HOW
Extracted from sales system logs automatically — reduces human error

02

EDA

EXPLORATORY DATA ANALYSIS

"Look at your data before you model it."

EDA is the practice of using statistics and visualisations to understand your data's structure, patterns, and problems.

WHY DO EDA BEFORE MODELLING?

1

🔍 See What Numbers Can't Show

A correlation of 0.5 tells you little. A scatter plot tells you everything. Patterns, clusters, anomalies — only visible visually.

2

🧠 Build Intuition About Your Data

You need to KNOW your data — its range, its gaps, its outliers — before you trust any model trained on it.

3

🧹 Guide Data Cleaning

EDA reveals WHERE the problems are: missing values, wrong types, duplicates, impossible values. Clean what matters first.

4

✅ Support Better Decisions

Which features actually matter? EDA tells you. Don't spend hours engineering useless variables.

5

📖 Tell a Story With Evidence

Good data science is storytelling. EDA gives you the evidence. Every chart is a sentence in your story.

UNIVARIATE ANALYSIS

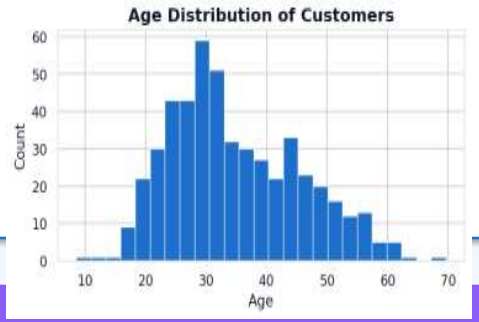
One variable at a time

📌 Analyse ONE variable at a time to understand its distribution, range, and typical values.

HISTOGRAM

Shows the distribution of a numeric variable — how values are spread across ranges.

- → Age distribution of customers
- → Price distribution of products
- → Income distribution



LINE CHART

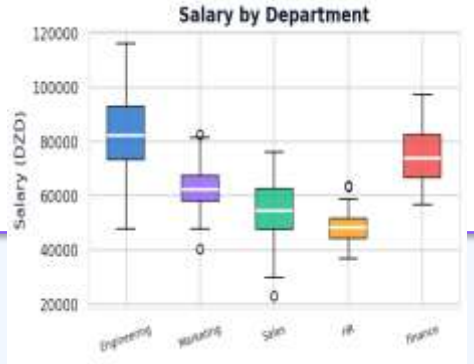
Shows how a variable changes over time — essential for time-series data.

- → Daily website visitors
- → Monthly sales revenue
- → Stock price history

BOX PLOT

Shows median, quartiles, and outliers at a glance — great for comparing groups.

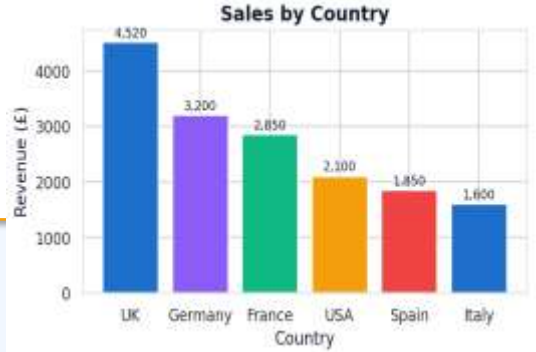
- → Salary by department
- → Score by student group
- → Height by age group



BAR CHART

Compares counts or values across categories — the most common chart.

- → Sales by country
- → Number of orders per day
- → Customers per age group



MULTIVARIATE ANALYSIS

Discovering relationships

📌 Analyse TWO or more variables together to find relationships, patterns, and correlations.

● SCATTER PLOT

Plots two numeric variables against each other. Reveals correlation direction and strength.

Use when: checking if variable A increases as B increases.

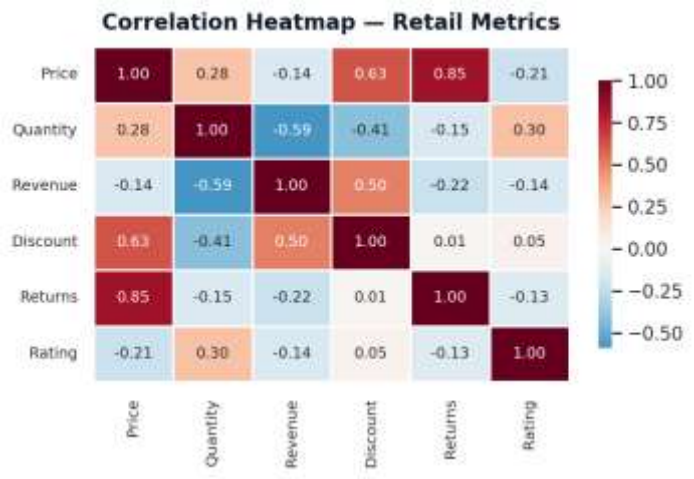
📌 Ad spend vs Revenue / Height vs Weight

▦ CORRELATION HEATMAP

Shows correlation between ALL numeric variables at once. Values from -1 to +1.

+1 = perfect positive / -1 = perfect negative / 0 = no correlation

📌 eCommerce metrics / Financial indicators



WHICH CHART TO USE? — QUICK DECISION GUIDE

YOUR QUESTION	BEST CHART	TYPE
How is this variable distributed?	Histogram / Box Plot	Univariate
How does this value change over time?	Line Chart	Univariate
How many items are in each category?	Bar Chart	Univariate
Is there a relationship between A and B?	Scatter Plot	Bivariate
How correlated are ALL my features?	Correlation Heatmap	Multivariate
How do distributions differ by group?	Grouped Box Plot	Multivariate
Compare 3 numeric variables at once?	Bubble Chart	Multivariate
What proportion does each category have?	Pie / Donut Chart	Univariate

EXERCISE — PICK THE RIGHT CHART

For each scenario below, decide which chart type to use and why.

1	A retail store wants to know which age group makes up the largest portion of its customers.	✓ Histogram / Bar Chart
2	You need to show how daily website visitors changed over the past year.	✓ Line Chart
3	An economist analyses the link between inflation, unemployment, and GDP growth.	✓ Scatter Plot + Correlation Heatmap
4	You need to check whether most employees earn similar salaries or if there are big outliers.	✓ Box Plot
5	You want to check if higher ad spend leads to higher revenue (monthly data).	✓ Scatter Plot

VISUALIZATION TOOLS — MATPLOTLIB & SEABORN

MATPLOTLIB

Foundation & Full Control

"The Swiss Army knife of Python plotting."

✅ Full low-level control over every element

✅ Works with NumPy arrays and lists

✅ Excellent for custom layouts and subplots

✅ Best for annotated, publication-ready charts

⚠️ More verbose — more lines of code

⚠️ Statistical plots need manual work

```
import matplotlib.pyplot as plt
```

`plt.plot(x, y)` / `plt.bar(x, y)` / `plt.hist(x)`

VS

SEABORN

Built on Matplotlib — Statistical Focus

"Beautiful statistical charts with less code."

✅ High-level statistical visualisations

✅ Works natively with Pandas DataFrames

✅ Beautiful defaults — less styling needed

✅ Heatmaps, violin plots, pair plots built-in

⚠️ Less flexible for custom chart layouts

⚠️ Must understand its opinionated API

```
import seaborn as sns
```

`sns.heatmap(df.corr())` / `sns.boxplot(x, y)`

03

CLEAN

DATA CLEANING

"Clean data = trustworthy AI."

Garbage in, garbage out.
No matter how good your model is,
dirty data produces wrong predictions.

WHAT IS DATA CLEANING?

Data cleaning is the process of detecting and correcting errors, inconsistencies, or missing information in a dataset — to make it accurate, consistent, and ready for analysis or ML.

? MISSING VALUES

PROBLEM:

Empty cells that most ML models cannot handle.

FIX:

```
Fill with mean/median
or remove the row.
```

ABC INCORRECT TYPES

PROBLEM:

Numbers stored as text. Dates as strings. Wrong format.

FIX:

```
Convert:
df['col'].astype(float)
pd.to_datetime()
```

DUPLICATES

PROBLEM:

Same row appears multiple times — distorts training data.

FIX:

```
df.drop_duplicates()
Keep first occurrence.
```

OUTLIERS

PROBLEM:

Values far outside normal range — may be errors or special cases.

FIX:

```
Investigate first.
Cap, transform, or
remove.
```

MISSING VALUES — HOW TO HANDLE THEM

WHY DOES DATA GO MISSING?

Survey not filled
User skipped a question

Sensor failure
IoT device stopped transmitting

Data entry error
Human forgot to fill the field

System migration
Old records lost during transfer

Private data
User opted out of sharing

Not applicable
Field doesn't apply to this row

1 DROP THE ROW

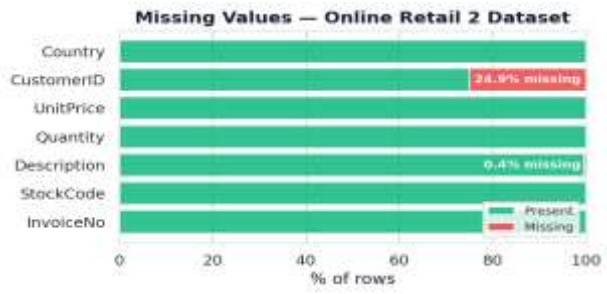
Best when: very few rows missing,
missing at random, enough data left.

```
df.dropna(subset=['Age'])
```

2 FILL WITH MEAN / MEDIAN

Best when: numeric column,
data is roughly symmetric.

```
df['Age'].fillna(df['Age'].mean())
```



Missing values visualised — Online Retail 2 Dataset

3 FILL WITH MODE

Best when: categorical column,
one value is dominant.

```
df['City'].fillna(df['City'].mode()[0])
```

DUPLICATES & INCORRECT DATA TYPES

DUPLICATES

What are they?

Rows that are identical or nearly identical — caused by data entry errors, system bugs, or merging datasets multiple times.

Why dangerous?

A model trained on duplicates will overfit to repeated data points, resulting in inflated performance metrics.

How to detect & remove:

```
df.duplicated().sum()        # count dups
df[df.duplicated()]          # see dups
df.drop_duplicates()         # remove all
df.drop_duplicates(['email'])  # by col
```

INCORRECT DATA TYPES

Common type problems:

❌ **Price stored as $99.99**

String with $ symbol — can't compute

❌ **Date as '2023-01-15'**

String — can't sort or do time math

❌ **'True'/'False' as strings**

Should be boolean — comparisons fail

❌ **ID stored as float**

CustomerID 1234.0 — comparison issues

Fix it with:

```
df['price'] = df['price'].str.replace('$','').astype(float)
df['date']  = pd.to_datetime(df['date'])
df['active']= df['active'].map({'True':True,'False':False})
df['id']    = df['id'].astype(int)
```

OUTLIERS — DETECT, DECIDE, HANDLE

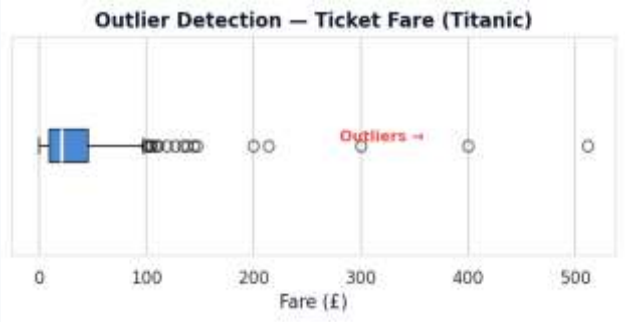
⚠ An outlier is a data point that falls far outside the normal range. It might be a genuine extreme value, a typo, or a sensor error. Always investigate before removing.

VISUAL METHOD
Box Plot

The box plot's whiskers extend to 1.5 × IQR. Any point beyond the whiskers is flagged as an outlier.

Use first — always visualise before computing.

```
sns.boxplot(data=df, x='salary')
```



Z-SCORE METHOD

Z-score measures how many standard deviations a value is from the mean. Values with |Z| > 3 are considered outliers.

Works best with normally distributed data.

```
from scipy import stats

df[np.abs(stats.zscore(df['salary'])) > 3]
```

OUTLIERS — Cholesterol value 500 is far outside normal range (180-240)

patient_id	age	blood_pressure	cholesterol	admission_date
1	45	-1.34	200	2021-05-01
2	44.33	0.45	210	2021-05-02
3	50	1.12	500	2021-05-03
4	38	-0.23	190	2021-05-04

IQR METHOD

IQR = Q3 − Q1 (middle 50% of data).
Lower fence = Q1 − 1.5×IQR
Upper fence = Q3 + 1.5×IQR

Most robust method — works for skewed data too.

```
Q1=df['col'].quantile(0.25)

Q3=df['col'].quantile(0.75)

IQR=Q3-Q1

df[(df['col']>Q1-
1.5*IQR)&(df['col']<Q3+1.5*IQR)]
```

THE DATA CLEANING PIPELINE

1 LOAD & INSPECT

```
df.shape / df.info() / df.head()
df.describe() / df.dtypes
```

2 FIND MISSING

```
df.isnull().sum()
df.isnull().mean() * 100
```

3 HANDLE MISSING

```
dropna() / fillna(mean)
fillna(mode) / forward fill
```

4 FIX DATA TYPES

```
astype() / pd.to_datetime()
str.replace() / map()
```

5 REMOVE DUPLICATES

```
df.duplicated().sum()
df.drop_duplicates()
```

6 DETECT OUTLIERS

```
sns.boxplot() / Z-score / IQR
Investigate → cap or remove
```

7 VALIDATE & SAVE

```
df.info() / df.describe() again
df.to_csv('clean_data.csv')
```

→ Always run df.info() and df.describe() BEFORE and AFTER cleaning to compare.

BEFORE vs AFTER — DATA CLEANING IN ACTION

✗ BEFORE CLEANING

date	category	price	rating
04/01/2023 PM	Electrncs	twenty	4
2023-04-02	Books	$34.35	5
2023/04/03	Clothing	49.99	4 stars
03-04-2023	Fod←Typo	9.99	10
2023-04-05T...	Furniture	20€	5

✓ AFTER CLEANING

date	category	price	rating
2023-04-01 12:00:00+00	Electronics	199.99	4
2023-04-02 00:00:00+00	Books	20.0	5
2023-04-03 00:00:00+00	Clothing	49.99	4
2023-04-04 00:00:00+00	Food	9.99	5 (capped)
2023-04-05 08:00:00+00	Furniture	20.0	5

🔑 Key rule: Uniform date format / Numeric prices (float) / Consistent categories / Ratings capped at valid range / No extra whitespace

SESSION 02 — KEY TAKEAWAYS

DATA

Every AI project starts with data. Always apply the WWWWW-H framework to understand your dataset before touching the code.

EDA

Never skip exploratory analysis. Visualise distributions, find patterns, and detect anomalies BEFORE modelling.

CHART CHOICE

Match your chart to your question. Distribution → Histogram. Trend → Line. Relationship → Scatter. All features → Heatmap.

CLEANING

Dirty data = wrong AI. Handle missing values, fix data types, remove duplicates, and investigate outliers every time.

PIPELINE

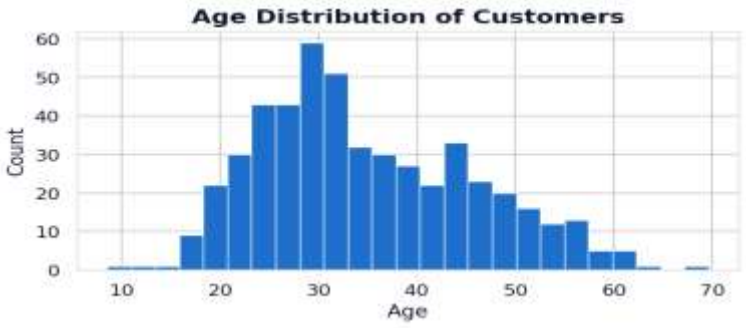
Always follow the 7-step cleaning pipeline: Load → Inspect → Missing → Types → Duplicates → Outliers → Validate.

MINDSET

80% of real AI work is data preparation, not modelling. Master this and you are already better than most beginners.

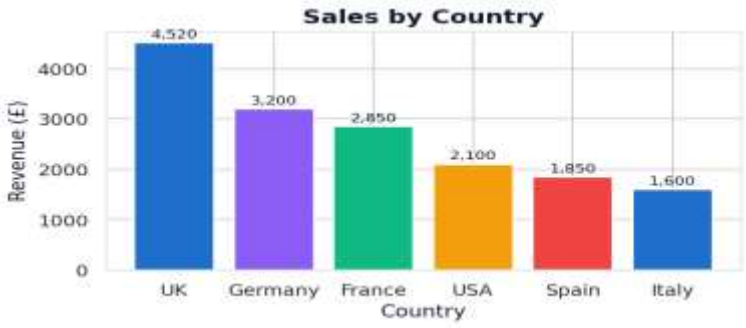
CHART GALLERY — ALL 6 ESSENTIAL PLOTS

HISTOGRAM



Distribution of one
numeric variable

BAR CHART



Compare categories
or groups

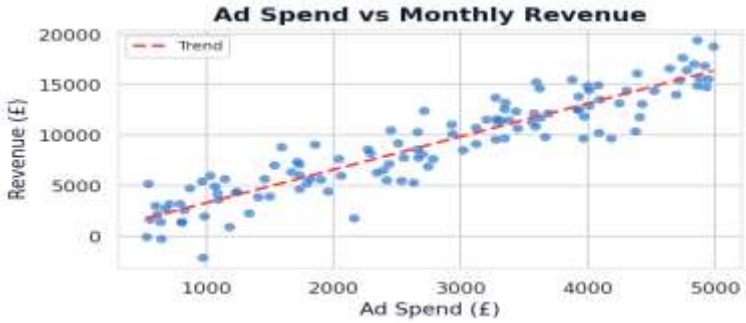
LINE CHART

Trend over time
(time series data)

BOX PLOT

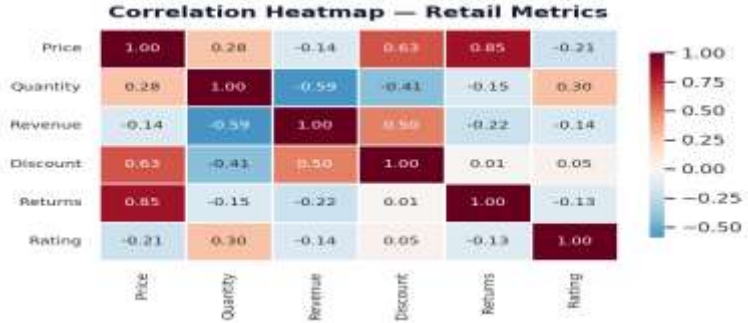
Distribution +
outlier detection

SCATTER PLOT



Relationship between
two numeric variables

HEATMAP



Correlation between
all variables at once

YOUR DATASET FOR THIS SESSION

Online Retail II Dataset

kaggle.com/datasets/mashlyn/online-retail-ii-uci

Source	UCI Machine Learning Repository / Kaggle
Rows	1,067,371 real transactions
Columns	8 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
Period	December 2009 – December 2011 (2 full years)
Origin	UK-based online gift retailer — customers worldwide
Why perfect?	Contains ALL data issues: missing CustomerID, negative quantities, duplicates, outliers in price

HOW TO DOWNLOAD

1. **Go to kaggle.com**
   Create a free account if you don't have one
2. **Search for:**
   Online Retail II UCI
3. **Click Download**
   Download the .xlsx file (~45MB)
4. **Open Jupyter**
   Load with pandas read_excel()
5. **Start exploring!**
   Run df.info() and df.describe()

```
import pandas as pd    df = pd.read_excel('online_retail_II.xlsx', sheet_name='Year 2009-2010')    print(df.shape)    df.head()
```